

BaiZe Technical Report

Abstract—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, and cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a ≈ 2.2 -billion-parameter agentic language model built *from scratch* on a fixed compute budget around a *hybrid* state-space backbone. BaiZe adopts a sequential hybrid architecture—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—whose linear-time decoding is well suited to inference-heavy, cost-sensitive agents, and is trained with the NVIDIA NeMo recipe on Megatron-Core. The architecture is chosen through a matched from-scratch comparison against a dense transformer counterpart, in which the hybrid attains lower loss and faster inference with fewer parameters at a modest training-throughput cost. We then run a staged, budget-constrained hyper-parameter search (S1–S5) on $8\times H100$ over the final hybrid backbone, which selects a stable learning rate of 1×10^{-3} under a warmup–stable–decay schedule with an 86:10:4 annealing mixture, reaching a validation loss of 2.2054 after 20000 steps and 2.6739 ± 0.0469 across three seeds at 5000 steps. We release the complete recipe and training command to support the reproducible construction of efficient agentic small language models.

Index Terms—small language model, agentic LLM, hybrid architecture, state-space model, budget-constrained search, efficient inference

I. INTRODUCTION

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints.

Recent small language models (SLMs, < 2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct, from scratch and on a strictly limited compute budget, an end-to-end multimodal agentic system—an efficient hybrid language core paired with a custom vision encoder, aligned into a semiconductor-domain agent—and (ii) to expose the concrete, reproducible decisions that lead to it.

a) *BaiZe*.: We present BaiZe, an end-to-end multimodal agentic system assembled from scratch across five stages: (i) pre-training a ≈ 2.2 B-parameter hybrid (state-space + attention) LLM, (ii) post-training it for agentic capability, (iii)

pre-training a custom vision encoder, (iv) aligning the two into a multimodal model, and (v) adapting the result to the semiconductor domain. Three design pillars organise our work:

- **Architecture.** A sequential hybrid language backbone—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—giving linear-time decoding for inference-heavy agents, paired with a from-scratch vision encoder.
- **Reduced-horizon proxy search.** We exploit short, cheap proxy runs to rank candidate hyper-parameters, so that the ranking discovered at small scale carries over to the full model without re-tuning at scale.
- **Budget-constrained stage-wise pipeline.** Each of the five stages is trained under a hard GPU-hour budget, with a staged search protocol (S1–S5) for the language backbone and a single frozen evaluation protocol shared by the later stages.

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bound training efforts in the EDA and embedded-AI communities.

II. BACKGROUND AND RELATED WORK

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [1] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [2] and Gemma [3] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [4] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [5] and Mamba-2 [6] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [7] and Hymba [8]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs under-documented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

C. Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA [9] established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision [10] consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section VII), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [11], [12]. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

III. ARCHITECTURE AND PIPELINE OVERVIEW

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [6], [7]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core’s Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the $\approx 2\text{B}$ scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. The full configuration is listed in Table I; the model totals $\approx 2.22\text{B}$ parameters.

TABLE I
BAIZE LANGUAGE-BACKBONE CONFIGURATION (MAMBA2-HYBRID).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	4096
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	BF16
Parameters	$\approx 2.22\text{B}$

B. Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from four iso-parameter candidates at the 500–600 M scale; Section VI reports the comparison and its outcome.

C. Pipeline and Global Environment

Table II summarises the five-stage pipeline. All stages run on $8 \times \text{NVIDIA H100 GPUs}$ (CUDA 12.8, PyTorch 2.8.0, Python 3.10); fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size = 1` and BF16 precision throughout.

TABLE II
BAIZE FIVE-STAGE PIPELINE OVERVIEW.

Stage	Task	Status	Section
(i)	LM pre-training	complete	IV
(ii)	Post-training (SFT, RLVR)	design	V
(iii)	Vision encoder	complete	VI
(iv)	MLLM alignment	design	VII
(v)	Domain adapt. (RLHF)	design	VIII

IV. LLM PRETRAINING

Stage (i) pre-trains the hybrid language backbone (Section III-A) from scratch under a fixed budget. This section covers the architecture-selection rationale, the training recipe, and the resulting measurements; the backbone configuration itself is given in Table I.

A. Architecture Selection Rationale

We adopt the hybrid backbone on three grounds. **(i) Inference efficiency.** State-space layers decode with constant-time recurrence per token, while the sparse attention layers retain long-range context; this directly targets the inference-heavy, cost-sensitive deployments that motivate BaiZe. **(ii) Controlled comparison.** A matched from-scratch comparison at the $\approx 2\text{B}$ scale between a dense transformer (MiniCPM5-2B) and Mamba2-hybrid—trained under identical data, seed,

and GPU footprint—favours the hybrid, which converges to a lower loss with fewer parameters ($\approx 2.22\text{B}$ vs. 2.51B , -11.6%) and decodes faster under our direct-drive harness, at a $\sim 24\%$ training-throughput cost (Table III). (iii) **Feasibility.** Megatron-Core ships a Mamba-hybrid provider whose 56-layer interleaving pattern we reuse verbatim, halving only the hidden and FFN widths to reach the $\approx 2\text{B}$ scale, making the backbone directly executable under the NeMo recipe without custom kernels.

B. Training Framework and Reduced-Horizon Search

We train with the NVIDIA NeMo pre-training entry point, driving the Megatron-Core [13] `NVIDIAMambaHybridModelProvider2B` provider directly with `tensor_model_parallel_size = 1`. To keep the entire sweep within budget, we run all hyper-parameter searches at a reduced iteration count and treat the resulting short-horizon ranking as predictive of the full-model optimum. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled to raise throughput.

C. Learning-Rate Schedule

BaiZe trains under a warmup–stable–decay-style learning-rate schedule whose hyper-parameters—warmup length, stable learning rate, and the decay ratio—are themselves swept in Section IV-D (cosine decay is kept as an alternative). The remaining schedule and batch-size settings (global batch size, sequence length, packing) are held fixed across configurations and re-measured on the final backbone, with figures reported where applicable.

D. Budget-Constrained Search Protocol

Because a full grid is infeasible under a fixed GPU budget, we adopt a staged, single-variable-at-a-time protocol:

- **S1 (baseline).** Reproduce the Mamba2-hybrid reference configuration (stable LR 3×10^{-4} , WSD) to establish a loss/throughput baseline.
- **S2 (learning-rate sweep).** Sweep the stable learning rate over seven points from 2×10^{-4} to 1×10^{-3} with WSD held fixed.
- **S3 (schedule axes).** On the winning learning rate, perturb one variable at a time—schedule family (WSD vs. cosine), decay ratio, warmup length, and `min_lr`.
- **S4 (annealing data mix).** Ablate the annealing data blend (pure Ultra-FineWeb-L3 vs. adding code and math) under the winning configuration.
- **S5 (long run + seeds).** Retrain the winning configuration from scratch over a 20000-step horizon, plus a three-seed short-horizon reproducibility check.

At every stage only a single variable changes relative to the running best, isolating each factor’s contribution while keeping total GPU-hours within budget. The architectural search space is validated up front: the hybrid (Mamba2-hybrid) and dense (MiniCPM5-2B) candidates are compared head-to-head from scratch (Section IV-A), which selects the hybrid backbone before the hyper-parameter search begins.

E. Data

Pre-training uses the Ultra-FineWeb-L3 English subset ($\approx 1.8\text{T}$ tokens), streamed and packed to a sequence length of 4096, tokenised with the DeepSeek-V4.1-Flash tokenizer (vocabulary size 129,281). Complementary corpora—code (UltraData-Code, 1.2T), mathematics (UltraData-Math, 515G), and an agentic instruction-tuning set (UltraData-SFT-Agent, 51G)—are reserved for post-training (Section V).

F. Search Budget

The pre-training search is bound by a ≈ 92 GPU-hour plan on eight H100 GPUs (≈ 192 GPU-hours wall-clock available); ≈ 85 GPU-hours were consumed. All short-horizon sweeps (S1–S4) run 5000 steps at a global batch size of $8 \times 4094 \approx 32.7\text{K}$ tokens/step ($\approx 164\text{M}$ tokens, ≈ 4.0 GPU-hours each); the verification long run (S5) runs 20000 steps ($\approx 655\text{M}$ tokens, ≈ 13.9 GPU-hours). Validation loss is reported at matched step counts so that comparisons across configurations remain apples-to-apples.

G. Results

The winning configuration is a stable learning rate of 1×10^{-3} under a warmup–stable–decay (WSD) schedule (warmup 250 steps, decay 500 steps, `min_lr` $= 1 \times 10^{-5}$), with an 86:10:4 blend of Ultra-FineWeb-L3, code, and math data in the annealing tail. Over the 5000-step horizon the validation loss reaches **2.7647**; the learning-rate sweep improves monotonically from 2.9037 (2×10^{-4}) to 2.7627 (1×10^{-3}). Each short-horizon ablation confirms a single-variable choice: WSD beats cosine (2.7679 vs. 2.8155), a 10% decay ratio beats 5% (2.7679 vs. 2.7900), a 250-step (5%) warmup beats a fixed 2000-step warmup (2.7679 vs. 2.7957), and `min_lr` $= 1 \times 10^{-5}$ narrowly beats 3×10^{-5} (2.7647 vs. 2.7679). Adding code and math to the annealing mix lowers the final loss monotonically (2.7621 pure L3 $\rightarrow$ 2.6793 L3+code $\rightarrow$ 2.6298 L3+code+math).

The 20000-step verification run reaches a final training loss of **2.2054**, with the decay tail (steps 19500–20000) annealing the loss from ~ 2.34 to 2.2054 (annealing gain ~ 0.14 ; Figure 1). Across three seeds the 5000-step loss is 2.6739 ± 0.0469 (mean $\pm$ sample standard deviation, $n=3$; per-seed 2.6402, 2.6539, 2.7275).

Table III contrasts the hybrid backbone against a dense transformer at the $\approx 2\text{B}$ scale, trained from scratch under matched data, seed, and GPU footprint; the hybrid offers fewer parameters and faster decoding at a modest training-throughput cost.

Table IV reports the hyper-parameter search outcomes at a matched horizon; the searched axes target the hybrid training recipe (stable learning rate, warmup length, decay ratio, and schedule family).

V. LLM POST-TRAINING

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe’s domain claim: the

TABLE III

ARCHITECTURE-LEVEL COMPARISON MOTIVATING THE HYBRID CHOICE (DENSE = MINICPM5-2B, HYBRID = BAIZE; MEASURED FROM SCRATCH UNDER MATCHED DATA, SEED, AND GPU FOOTPRINT). THE DECODE GAP IS MEASURED UNDER THE MEGATRON-CORE DIRECT-DRIVE HARNESS WITHOUT CUDA GRAPHS, WHERE THE DENSE BASELINE IS CPU-LAUNCH-BOUND, AND THEREFORE NARROWS UNDER PRODUCTION INFERENCE STACKS.

Metric	Dense	Hybrid	Edge
Parameters	2.51B	2.22B	hybrid (−11.6%)
Final loss (1k steps)	4.81	4.68	hybrid (−0.12)
Training tok/s	89K	72K	dense (+24%)
Prefill tok/s	21.4K	23.7K	hybrid (+10.5%)
Decode tok/s	2.2	22.8	hybrid (+10.3×)

TABLE IV

HYPER-PARAMETER AND DATA-MIX SEARCH OUTCOMES AT A MATCHED 5000-STEP (S5: 20000-STEP) HORIZON.

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline ($lr=3\times10^{-4}$, WSD)	2.849	baseline
S2-07	stable LR = 1×10^{-3}	2.7627	best LR
S3-02	schedule family: cosine	2.8155	WSD retains
S3-03	decay ratio 5%	2.7900	10% retains
S3-04	warmup 2000 steps	2.7957	250 retains
S4-03	anneal L3+code+math 86:10:4	2.6298	best mix
S5-01	winner, 20000 steps	2.2054	winner

pre-training corpus of Section IV-E is general web, code, and mathematics text and contains no EDA artifacts, so the domain prior must be introduced here. We organise the stage as three steps—*domain annealing*, *supervised fine-tuning*, and *reinforcement learning*—all evaluated under a single frozen protocol (Section V-D).

A. Domain Annealing

The cheapest lever on the domain prior is to continue pre-training on EDA text before any behavioural tuning. We reuse the warmup–stable–decay machinery of Stage (i) and anneal on a corpus assembled from PyAether and SKILL API references, EDA tool documentation, and open HDL sources,

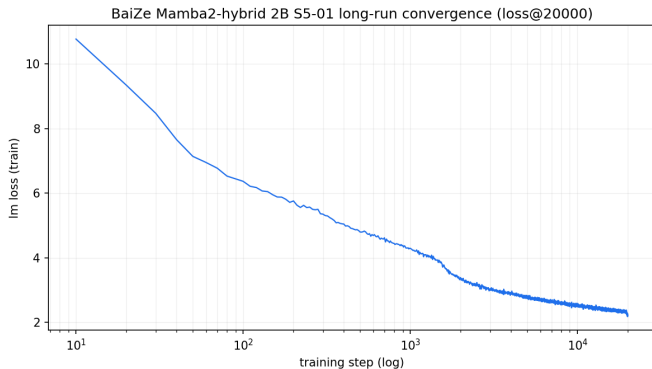


Fig. 1. Training-loss curve of the winning Mamba2-hybrid configuration over the 20000-step verification run (seed 1234); the decay tail (steps 19500–20000) anneals the loss from ~ 2.34 to 2.2054.

blended with the general mixture at a controlled ratio. Because the step changes only the data distribution, we contrast it against an SFT-only control to isolate its contribution from the behavioural tuning that follows.

B. Supervised Fine-Tuning

SFT teaches the generate–execute–refine loop that the deployment target requires. The corpus combines the agentic instruction set (UltraData-SFT-Agent, 51 G) with PyAether-specific material synthesised from API documentation. We ablate the data mixture (agentic / code / mathematics / PyAether), the presence of interleaved tool-call traces, full fine-tuning versus LoRA, and the sequence length: agentic traces—code, tool returns, and successive refinements—routinely exceed the 4096-token window of Table I, so window extension is treated as a first-class variable rather than a fixed setting.

C. Reinforcement Learning with Verifiable Rewards

An execution sandbox turns correctness into a verifiable signal, so we optimise with group-relative policy optimisation and no value network. Table V separates the reward variants we study. A binary signal is the simplest baseline; partial credit over executable attempts addresses the sparsity of pass/fail; a tool-call penalty aligns the policy with the harness cost model; and an API-legality term penalises hallucinated symbols. Because the reward is computed from assertions a policy could in principle target directly, we keep the harness anti-cheat hook enabled in every run and report a cheat rate beside every accuracy figure.

TABLE V

REWARD VARIANTS STUDIED IN STAGE (II).

ID	Reward
R1	binary pass/fail
R2	executable + assertion pass ratio
R3	R2 + tool-call penalty
R4	R3 + API-legality term

D. Evaluation Protocol

All post-training variants are scored on **EDA-Eval-PyAether** [14], a benchmark of 158 real-world EDA scripting tasks. Each task pairs a natural-language prompt and a function signature with assertion-based test code, and is scored by execution in the vendor sandbox: a task passes when the generated code runs without error and every assertion holds. We follow the benchmark’s own protocol so that our numbers are comparable to published ones: five independent runs with reshuffled task order and fresh context, reporting mean Pass@1 and its run-to-run standard deviation, with a 2500-second per-task timeout and a single non-restartable trace per task.

Two controls make the comparison interpretable. First, *contamination*: no task, assertion, or reference solution from the benchmark enters any training split; PyAether training material is drawn only from API documentation, and we

isolate it by symbol so that benchmark targets cannot leak. Second, *harness parity*: only the backbone is varied. The retrieval stack, sandbox, anti-cheat hook, and agent scaffold are held byte-identical to the published configuration, because the harness supplies most of the observed accuracy and a variant harness would confound the model’s contribution with the scaffold’s.

Results are reported as [TBD].

VI. VISION ENCODER PRETRAINING

Stage (iii) pre-trains a custom vision encoder from scratch for the multimodal stage (Section VII). This section presents the architecture-selection rationale, the pre-training recipe, and the resulting measurements.

A. Architecture Selection

We compare four candidate vision encoders at an iso-parameter 500–600 M scale, all *randomly initialized* (no upstream ImageNet/CLIP/SigLIP weights) so that only the architecture body is contrasted. All candidates are trained under a unified stack: `open_clip` 3.2.0, a frozen shared text tower (12 layers, width 512, 8 heads, one BPE for all towers), the SigLIP sigmoid-contrastive objective, and a unified 500K subset of LLaVA-OneVision-1.5 (imagenet/EN), at resolution 224 / patch 16 / bf16, learning rate 10^{-3} (AdamW, warmup 100 + cosine), seed 1234, batch 32×6 . The four candidates are:

- **OpenVision2** — a pure-attention ViT (width 1024, depth 30, heads 16, MLP 4096), 505.0 M parameters.
- **MambaEye** — a pure visual state-space model (Vim-style bidirectional SSM, width 1024, depth 40, $d_{\text{state}}=16$, expand 2), 534.9 M.
- **MoE-ViE** — a sparse Mixture-of-Experts encoder (8 experts $\times$ ff 512, top-2 routing), 505.2 M total / 222.0 M active per token.
- **DeepEncoderV2** — an Attention+SSM hybrid (Alternating Attention Mamba blocks), 516.9 M.

Table VI summarizes the measured parameter counts and the three hard metrics (loss, training throughput, inference latency), all measured under DP6 on $6 \times \text{H100}$ with matched batch and seed.

TABLE VI

ARCHITECTURE-LEVEL COMPARISON FOR THE VISION ENCODER (FROM-SCRATCH, UNIFIED OPEN_CLIP + SIGLIP + FIXED TEXT TOWER). INFERENCE LATENCY IS MEASURED AT BATCH=1 / BF16 / 224/16. TYPE: ATTN = ATTENTION, A+S = ATTENTION + SSM; MOE-ViE ACTIVATES 222.0M PARAMETERS PER TOKEN.

Arch.	Params	Type	Loss@5k	img/s	ms/img
OpenVision2	505.0M	Attn	4.456	2139	6.51
DeepEncoderV2	516.9M	A+S	4.466	1459	17.76
MambaEye	534.9M	SSM	4.470	790	hang@bs1
MoE-ViE	505.2M	MoE	4.466	852	141.7

Finding. Over 5000–10000-step horizons all four architectures converge to ≈ 4.45 – 4.47 loss, with inter-architecture

differences < 0.02 (within multi-seed variance). Loss is therefore *architecture-invariant* at this scale and token count: at 224/16 the encoder produces only 196 tokens, too short for the SSM recurrence or MoE routing to amortize their per-token overhead into a quality gain. Consequently the architecture choice is driven solely by efficiency, where the pure-attention **OpenVision2** dominates— $1.47\times$ the training throughput of the next-best (DeepEncoderV2) and $2.7\times$ lower inference latency, and full-SSM MambaEye even stalls at batch 1.

Methodological note. The apparent “loss plateau” of 6.7342 observed in a 1000-step sweep is an artifact of the cosine schedule collapsing the learning rate to $\approx 10^{-5}$ by $\sim$ step 700; extending the cosine span to 5000–10000 steps lets the loss continue to ≈ 4.45 . Sub-1k-step comparisons of these encoders are therefore uninformative.

B. Configuration and Ablations

Holding the winner (OpenVision2) fixed, we sweep the core degrees of freedom single-variable-at-a-time (each 3000 steps, avoiding the LR-collapse regime): resolution/patch (Table VII), objective function (Table VIII), and learning rate, plus a multi-seed robustness check and a held-out retrieval proxy.

TABLE VII
RESOLUTION/PATCH ABLATION ON OPENVISION2 (ANCHOR 224/16).

res/patch	Loss	Train img/s
224/16 (anchor)	4.9962	1582.1
336/16	4.9962	1630.1
448/16	4.9962	960.4
224/14	4.9962	1613.8
336/14	4.9962	1289.7
448/14	4.9962	757.3

TABLE VIII
OBJECTIVE-FUNCTION (SIGLIP VS CLIP INFONCE) AND LEARNING-RATE ABLATION ON OPENVISION2.

Variant	Loss	Note
SigLIP (anchor)	4.9962	sigmoid contrastive
CLIP InfoNCE	2.8941	softmax contrastive (scale differs)
lr 10^{-3} (anchor)	4.9962	—
lr 3×10^{-3}	4.4562	—
lr 5×10^{-3}	4.4542	—

Downstream validation is a zero-shot text $\leftrightarrow$ image retrieval proxy ($R@1/5/10$ on a held-out 5K subset from laioncn/EN, disjoint from the training source), as no ImageNet-style class labels are available for top-1 or linear probing. Across the ablations (each 3000 steps, anchor $lr=10^{-3}$), the resolution/patch sweep leaves loss essentially unchanged (≈ 4.996 at every resolution/patch) because the anchor learning rate has already collapsed to its minimum by step 3000 and pins the loss at that floor; the informative signal is throughput, which falls monotonically with the token count (1582 $\rightarrow$ 757 train img/s from 224/16 to 448/14). The objective switch (CLIP InfoNCE) yields a different loss scale

(2.894 vs 4.996, not directly comparable absent negative-logit calibration). The learning-rate sweep is decisive: raising lr from 10^{-3} to $3 \times 10^{-3} / 5 \times 10^{-3}$ lowers the final loss from 4.996 to 4.4562 / 4.4542, confirming that the anchor plateau is a learning-rate-collapse artifact rather than a data ceiling. The winning configuration remains **OpenVision2** at 224/16, SigLIP, $lr=10^{-3}$ (with the caveat that $lr=3 \times 10^{-3}$ reaches a lower loss at the same step count); the full reproduction commands are in the companion result report.

VII. MLLM ALIGNMENT

Stage (iv) connects the vision encoder of Section VI to the language backbone through a lightweight projector, following the staged curriculum established for open multimodal assistants: projector pre-training, mid-training, and instruction tuning. The design is motivated by the observation that the dominant EDA artifacts are visual—layout and schematic views account for 70.2% of the tasks in the evaluation benchmark [14]—so a multimodal core can be scored on the same execution-based protocol as the text-only one.

a) Task construction.: We build three task families, each chosen because it admits an objective ground truth. (i) *image*→*script*: given a rendered layout or schematic view, produce the PyAether or SKILL script that recreates it; supervision is available because executing a script yields its own reference rendering. (ii) *script*→*image*: predict the geometry that a given script produces, again self-labelled by execution. (iii) *layout VQA*: answer attribute questions—layer identity, device type, spacing and rule violations—labelled by domain experts. The first family is the strongest of the three: its output is a script, so it is scored by the same assertion sandbox used in stage (ii), which gives the multimodal and text-only stages a common metric instead of two incommensurable ones.

b) Curriculum and ablations.: We run stage-1 projector pre-training on caption-style pairs, stage-1.5 mid-training on the three families above, and stage-2 instruction tuning on the full mixture. The ablations vary the projector (MLP versus query-based), whether the vision tower is frozen or updated, and the number of visual tokens per image. The last is of particular interest. The architecture study of Section VI found image resolution immaterial at 224/16, where an image becomes only 196 tokens—a regime in which the training objective is dominated by coarse semantics and the encoder differences we measured were within noise. Layout and schematic views are the opposite: they are detail-dense line drawings whose meaning lives in fine geometry. We therefore treat resolution and visual-token count as first-order variables here rather than inheriting the earlier null result, which we read as a property of the proxy task and not of the encoder.

Results are reported as [TBD].

VIII. SEMICONDUCTOR DOMAIN ADAPTATION

Stage (v) completes the pipeline by adapting the multimodal model to proprietary design practice. It differs from stage (ii) in two respects, which we state explicitly because otherwise the two stages would coincide: the reward is no longer

verifiable but *learned from human judgement*, and the target behaviour is no longer general API usage but a team’s internal conventions—naming rules, layer policies, review checklists—which no public benchmark measures.

a) Method.: We collect pairwise judgements from in-house design reviews over the stage-(iv) task families, fit a domain reward model on them, and optimise the aligned model against that model under a KL-regularised objective. Because preference data is far scarcer than the verifiable signal of stage (ii), we initialise from the stage-(iv) checkpoint and constrain the update rather than training the objective from the pre-trained model.

b) Why a small model.: The rationale for a $\approx 2.2\text{B}$ core is strongest here, and it is a constraint rather than a preference. Proprietary layouts and proprietary preference data cannot leave the premises, so the choice is not between a large hosted model and a small local one—it is between a small local model and no model at all. Serving cost then dominates, and for an agentic session the relevant cost is not throughput on a short prompt but memory that grows with accumulated context: in the hybrid backbone of Section III-A, the four attention layers are the only component whose serving state grows with the session, while the Mamba-2 layers hold a constant-size recurrent state.

c) Evaluation.: We evaluate on held-out review items using expert agreement, and report the result beside the execution-based numbers of stage (ii) so that verifiable and preference-based adaptation can be compared on the same system rather than in separate accounts.

Results are reported as [TBD].

IX. CONCLUSION

We presented BaiZe, an end-to-end multimodal agentic system: a $\approx 2.2\text{B}$ -parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyperparameter search (S1–S5) on the final backbone. Stages (ii) to (v) are specified here as a protocol and evaluated on a common execution-based benchmark [14]; their measurements are reported as [TBD]. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the UltraData-SFT-Agent corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

REFERENCES

- [1] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, “Minicpm: Unveiling the potential of

small language models with scalable training strategies,” *arXiv preprint arXiv:2404.06395*, 2024.

- [2] A. Yang, B. Li, B. Zhang *et al.*, “Qwen3 technical report,” 2025.
- [3] G. Team, “Gemma 3 technical report,” 2025.
- [4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, “Agentbench: Evaluating llms as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [5] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [6] T. Dao and A. Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [7] NVIDIA, “Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models,” 2025.
- [8] X. Dong, Y. Fu, S. Diao *et al.*, “Hymba: A hybrid-head architecture for small language models,” 2024.
- [9] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, “Llava-onevision: Easy visual task transfer,” 2024.
- [11] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, “Chipnemo: Domain-adapted llms for chip design,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [12] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, “Chipbench: The llm chip design benchmark,” 2024.
- [13] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.
- [14] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, “Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration,” 2026.